



PROYECTO TERCERA EVALUACIÓN

PROGRAMACIÓN

Repositorio GitHub

<https://github.com/CarmenAlcazarDAM/ProyectoRefugio.git>

Carmen María Alcázar Martín

IES. Francisco de los Ríos

ÍNDICE

1. DESCRIPCIÓN DEL PROYECTO

- 1.1. Explicación del proyecto: definición de la solución elegida y en qué consiste.
- 1.2. Objetivo general y utilidad: justificación del proyecto enfocado a un entorno real de producción y su utilidad práctica.

2. FASE DE DISEÑO

- 2.1. Diseño de la Base de Datos:
 - 2.1.1. Modelo Entidad/Relación.
 - 2.1.2. Modelo Relacional
 - 2.1.3. Archivo SQL y Esquema: código de creación de tablas.
 - 2.1.4. Integridad y Consistencia: explicación del control de integridad referencial, claves primarias/foráneas, índices y restricciones aplicadas.
- 2.2. Diseño de Clases
 - 2.2.1. Diagrama de Clases.
 - 2.2.2. Diagrama de Comportamiento: casos de uso.
- 2.3. Diseño de la Interfaz Gráfica
 - 2.3.1. Prototipo de Pantallas: explicación del diseño planteado en JavaFX.

3. FASE DE DESARROLLO Y ARQUITECTURA

- 3.1. Estructura y Tecnologías Utilizadas: detalles del entorno de desarrollo.
- 3.2. Configuración del Proyecto y Persistencia.
 - 3.2.1. Conexión a la Base de Datos
- 3.3. Planificación y Temporalización: sprints de desarrollo establecidos y calendario de cumplimiento.

4. LISTA DE COMPROBACIÓN TÉCNICA

- 4.1. Aplicación de la Programación Orientada a Objetos: cómo se implementa la herencia, las restricciones con abstract y las llamadas a los constructores de la clase padre.
- 4.2. Relaciones del Modelo de Datos: detalle de las relaciones utilizadas cumpliendo el mínimo de 4 tablas.
- 4.3. Implementación del Patrón DAO: explicación de la construcción de los DAO y quién asume el control en las relaciones N:M.
- 4.4. Operaciones CRUD Básicas: mecanismo y flujo de las funciones Mostrar, Insertar y Eliminar.
- 4.5. Operaciones CRUD Avanzadas: mecanismo de actualización y control de impedancia Relación-Objeto.

5. VALORES AÑADIDOS Y CALIDAD DEL CÓDIGO

- 5.1. Validaciones y Control de Errores: control de campos protegidos, controles desactivados y muestra de mensajes de error gráficos.
- 5.2. Adaptabilidad de la Interfaz: ventanas redimensionables y comportamiento adaptativo.
- 5.3. Características Avanzadas (Streams y Lambdas): uso de características modernas de Java en el tratamiento de datos.
- 5.4. Gestión de Datos Complejos: tratamiento de fechas, expresiones regulares o datos multimedia.

6. CONSIDERACIONES FUTURAS Y CONCLUSIÓN

- 6.1. Aspectos a mejorar de cara a una futura versión.
- 6.2. Conclusiones y aprendizaje.

1. Descripción del Proyecto

1.1. Explicación del proyecto: definición de la solución elegida y en qué consiste.

Este proyecto se centra en el diseño y desarrollo de una aplicación de escritorio (standalone), orientada a optimizar y digitalizar la gestión integral de un refugio de animales. El término "*standalone*" refiere a cualquier sistema, programa o dispositivo que funciona por sí mismo, sin necesidad de depender de otros elementos, redes u otros equipos para operar.

Este software recoge en una única interfaz gráfica el control de dos de los pilares fundamentales de la organización: los animales en adopción y el equipo de voluntarios.

Gracias a un sistema gestor de persistencia de datos, la aplicación permite realizar un seguimiento exhaustivo de la estancia de los animales en el centro (ingresos, bajas, adopciones, etc), así como la organización de los datos de contacto de los adoptantes y las tareas realizadas en el refugio por parte de los voluntarios en los diferentes cheniles y jaulas donde descansan estos animales.

La interfaz se ha diseñado intentando conseguir la máxima accesibilidad y fluidez para que todo voluntario pueda usarla de forma intuitiva independientemente de su nivel de conocimiento informático. Además, se han implementado controles de seguridad para evitar la introducción de datos erróneos y garantizando una experiencia de uso cómoda.

1.2. Objetivo general y utilidad: justificación del proyecto enfocado a un entorno real de producción y su utilidad práctica.

En el ámbito real, las protectoras y refugios de animales suelen enfrentarse a problemas de organización debido al uso de herramientas fragmentadas (como, por ejemplo, la coordinación de los voluntarios a través de un grupo de WhatsApp) o registros manuales en papel y hojas de cálculo. Esto provoca pérdida de información, dificultades para cumplir con la trazabilidad legal de los animales y falta de eficiencia en la coordinación del voluntariado.

Por ello, este proyecto se ha desarrollado pensando en un entorno real de producción, persiguiendo los siguientes objetivos prácticos:

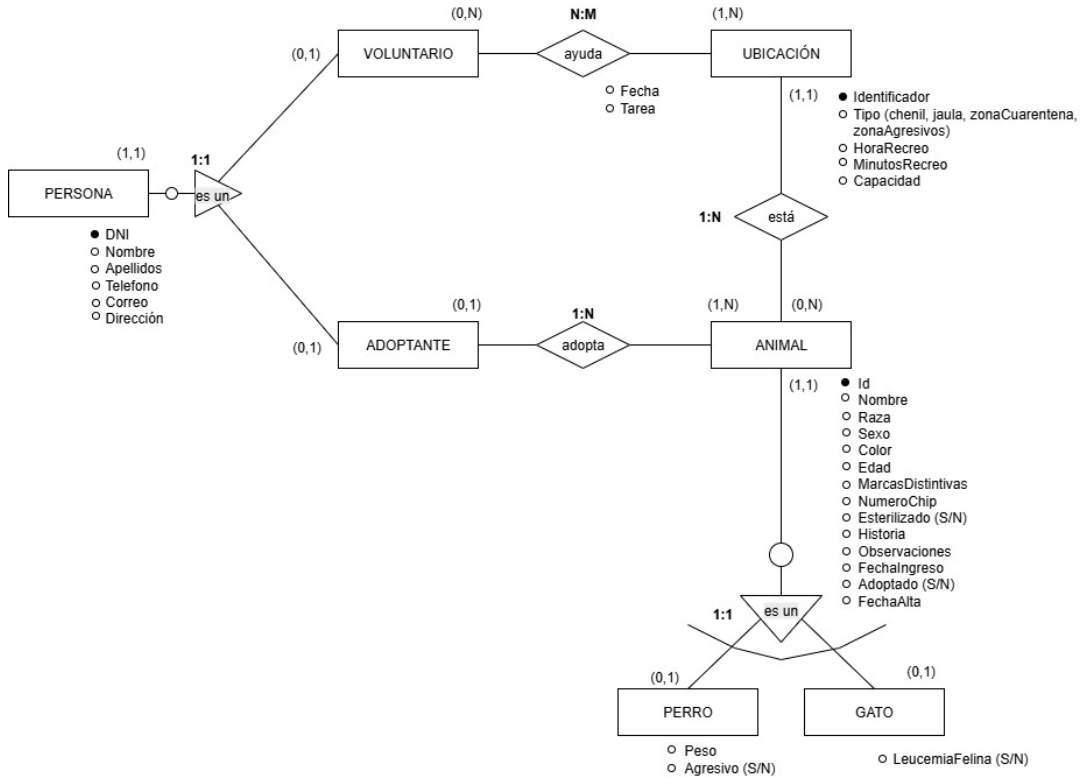
- Centralización de la información: sustituir los flujos de trabajo obsoletos por un sistema centralizado que garantice la integridad de los datos.

- Automatización de la gestión de estados: controlar de forma estricta cuándo un animal está físicamente en el refugio (gestionando campos de alta y bajas) para agilizar las consultas del personal.
- Mejora de la coordinación del voluntariado: manteniendo un registro actualizado de las tareas realizadas por los voluntarios y asociarlas a los distintos espacios del refugio.
- Reducción de errores administrativos: implementar validaciones y controles automáticos que minimicen la introducción de datos erróneos y mejoren la fiabilidad del sistema.
- Optimización del tiempo de gestión: agilizar procesos administrativos y de consulta mediante una interfaz intuitiva y accesible, permitiendo dedicar más tiempo al cuidado de los animales.

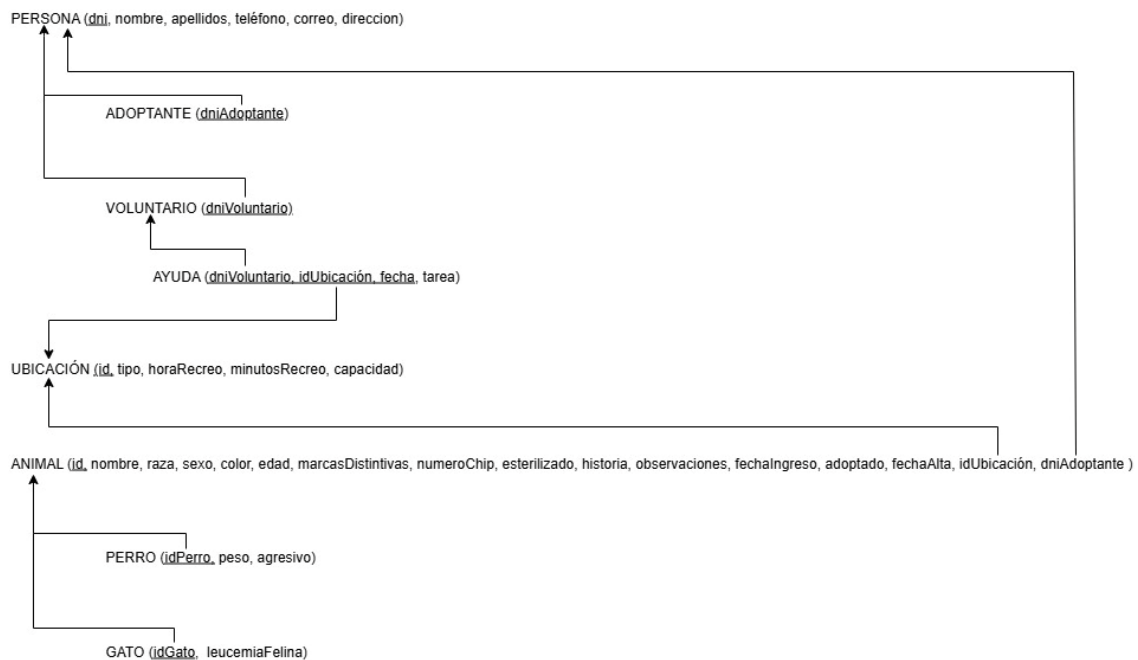
2. FASE DE DISEÑO

2.1. Diseño de la Base de Datos

2.1.1. Modelo Entidad/Relación



2.1.2. Modelo Relacional



2.1.3. Archivo SQL y Esquema: código de creación de tablas.

En este apartado veremos las sentencias SQL ejecutadas en MySQL Workbench para construir la arquitectura relacional de la base de datos.

```
CREATE TABLE persona (
    dni char(9) primary key,
    nombre varchar(20) not null,
    apellidos varchar(30),
    telefono varchar(12) not null,
    correo varchar(100),
    direccion varchar(100)
);

CREATE TABLE voluntario(
    dniVoluntario char(9) primary key,

FOREIGN KEY (dniVoluntario) references persona(dni)
);

CREATE TABLE adoptante(
    dniAdoptante char(9) primary key,

Foreign key (dniAdoptante) references persona(dni)
);

CREATE TABLE ubicacion(
    id int(2) unsigned primary key
    auto_increment,
    tipo
    enum('chenil','jaula','cuarentena','agresivos'),
    horaRecreo time,
    minutosRecreo int(3),
    capacidad INT(2) DEFAULT 1
);

CREATE TABLE animal(
    id int(4) unsigned primary key auto_increment,
    nombre varchar(20) not null,
    raza varchar(100) not null,
    sexo enum('hembra','macho') not null,
    color varchar (100) not null,
    edad varchar (20),
    marcasDistintivas text,
    numeroChip varchar(15),
    esterilizado boolean default 0,
    historia longtext,
    observaciones mediumtext,
    fechaIngreso DATE not null,
    adoptado boolean default 0,
    fechaAlta DATE default null,
    dniAdoptante char(9) null,
```

```

        idUbicacion int(2) unsigned not null,

FOREIGN KEY (dniAdoptante) references adoptante(dniAdoptante) on
delete no action on update cascade,

FOREIGN KEY (idUbicacion) references ubicacion(id) on delete no
action on update cascade
    );

CREATE TABLE perro(
    idPerro int(4) unsigned primary key,
    peso double(4,2) unsigned not null,
    agresivo boolean default 0,

FOREIGN KEY (idPerro) references animal (id) on delete cascade on
update cascade
    );

CREATE TABLE gato(
    idGato int(4) unsigned primary key,
    leucemiaFelina boolean default 0,
FOREIGN KEY (idGato) references animal (id) on delete cascade on
update cascade
    );

CREATE TABLE ayuda(
    dniVoluntario char(9),
    idUbicacion int(2) unsigned,
    fecha date not null,
    tarea text,
PRIMARY KEY (dniVoluntario, idUbicacion, fecha),

FOREIGN KEY (dniVoluntario) references voluntario(dni) on delete no
action on update cascade,

FOREIGN KEY (idUbicacion) references ubicacion(id) on delete no
action on update cascade
    );

```

2.1.4. Integridad y Consistencia: explicación del control de integridad referencial, claves primarias/foráneas, índices y restricciones aplicadas.

En este apartado se detalla el conjunto de reglas y mecanismos aplicados sobre el diseño de la base de datos en MySQL para garantizar que la información almacenada se mantenga precisa, válida y consistente en todo momento.

- **Claves primarias (PRIMARY KEY):** he utilizado dos formas de identificación únicas

- Claves Subrogadas (AUTO_INCREMENT): usado en tablas independientes como `ubicacion` y `animal`, delegando en la base de datos la generación de un identificador numérico único.
- Claves Compartidas (Herencia): en las tablas especializadas (`voluntario`, `adoptante`, `perro` y `gato`), la clave primaria es a su vez una clave foránea que apunta a la tabla padre (`persona` y `animal`). Esto garantiza una relación estricta de 1:1 la jerarquía.
- **Claves Foráneas** (FOREIGN KEY): establecen los vínculos de integridad referencial entre las distintas entidades del refugio. Impiden la inserción de registros que apunten a datos inexistentes (por ejemplo, asignar un `animal` a un `dniAdoptante` que no esté dado de alta previamente).

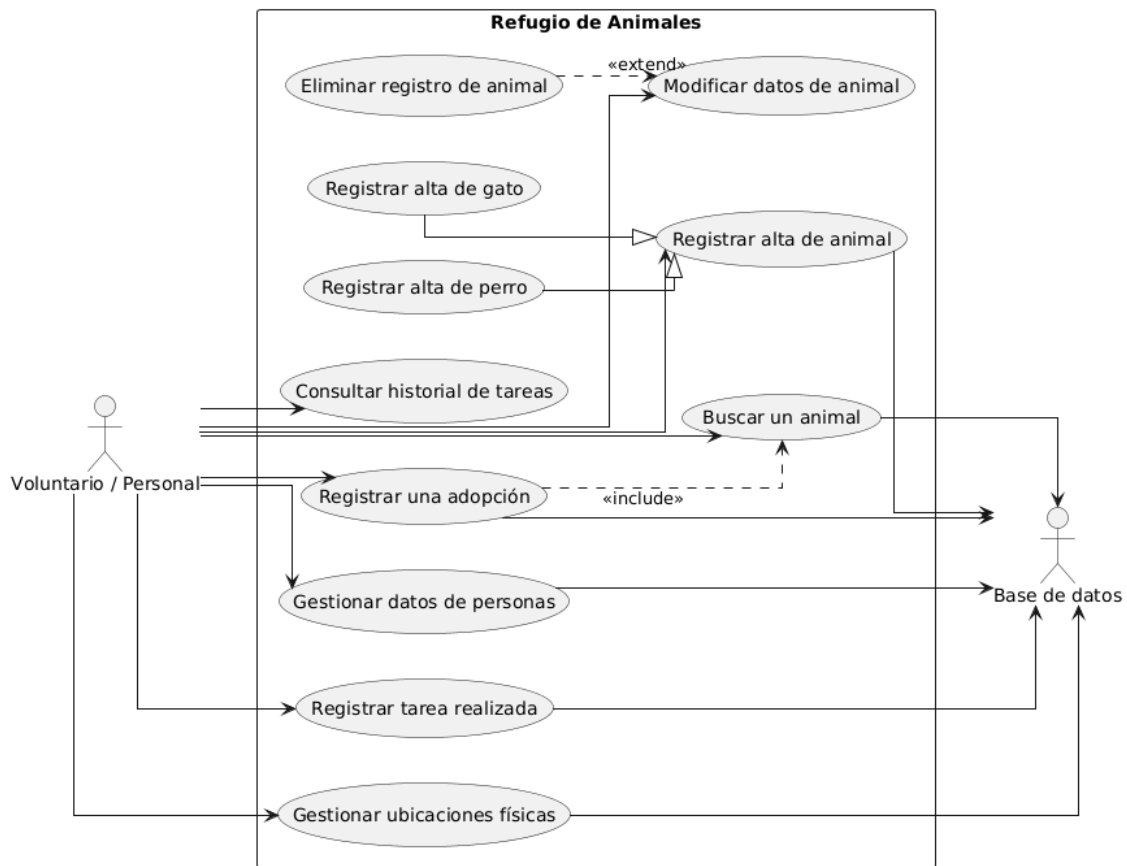
Para gestionar el comportamiento del sistema ante la modificación o eliminación de registros vinculados, se han configurado políticas específicas empleando el motor InnoDB:

- **Eliminación en Cascada** (ON DELETE CASCADE): aplicada exclusivamente en las relaciones de herencia. Si un `animal` es borrado de base de datos, su registro en la tabla hija `perro` o `gato` se elimina automáticamente, evitando inconsistencias en el sistema.
- **Restricción de Borrado** (ON DELETE NO ACTION): aplicada en relaciones operacionales críticas, como la vinculación de un `animal` con un `adoptante`. Esto impide la eliminación de un `adoptante` si existen registros históricos asociados a ellos.
- **Actualización en Cascada** (ON UPDATE CASCADE): configurada en todas las claves foráneas. Si se modifica un identificador principal (como el `dni` de una `persona`), el cambio se propaga de forma automática a todas las tablas dependientes (`voluntario`, `adoptante`, `animal`, `ayuda`).

Las restricciones aplicadas son las siguientes:

- **Restricción de Obligatoriedad** (NOT NULL): aplicada en atributos esenciales (como `nombre`, `sexo`, `color`, `fechaIngreso` o `idUbicacion` en la tabla `animal`), obligando a la aplicación a proporcionar datos válidos antes del guardado en el sistema.

2.2.2. Diagrama de Comportamiento: casos de uso



2.3. Diseño de la Interfaz Gráfica

2.3.1. Prototipo de Pantallas: explicación del diseño planteado en JavaFX.

Para el desarrollo del prototipo de pantallas en JavaFX he optado por un diseño intuitivo, orientado a la eficiencia del usuario (personal y voluntarios del refugio). El objetivo principal es que la gestión diaria de los animales, ubicaciones y personas suponga el menor esfuerzo para el usuario, manteniendo una interfaz limpia y coherente.

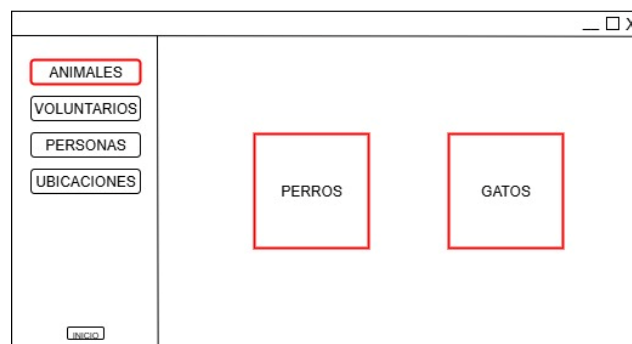
En lugar de abrir múltiples ventanas flotantes que saturen el escritorio del usuario, la aplicación utiliza una estructura basada en un contenedor dinámico. Los únicos dos casos de ventanas flotantes es para el registro de los animales y las personas, ya que es necesario introducir un mayor número de datos.

Aspectos importantes a tener en cuenta:

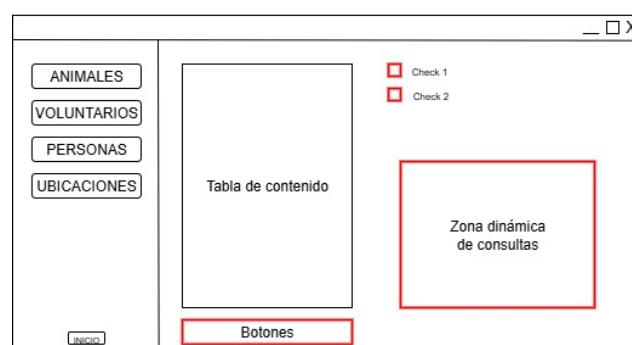
- La pantalla principal (PrincipalController) actúa como el marco global de la aplicación. Contiene un menú de navegación lateral fijo con botones claros para cada sección básica (Animales, Voluntarios, Personas, Ubicaciones). Al pulsar cualquier botón del menú, el método `SceneManager.cargarVista()` reemplaza dinámicamente el contenido del panel central (`panelContenido`).



- Subnavegación: la sección de animales no mezcla los tipos de animales en una sola lista confusa, sino que presenta dos botones, uno para seleccionar Perro, y otro, para seleccionar Gato.

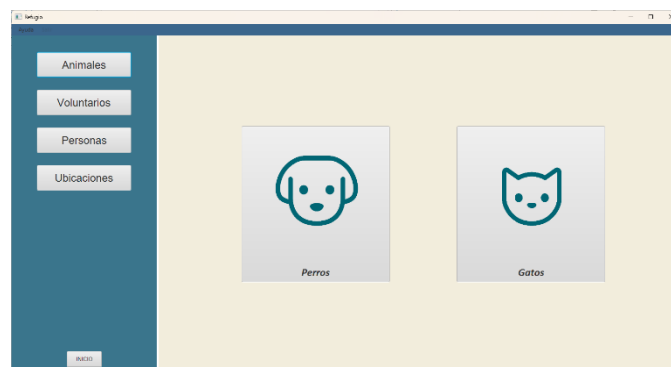


- Formularios Adaptativos: se reutiliza la misma interfaz visual adaptando sus campos en tiempo de ejecución según el contexto. Por ejemplo, si se registra un perro, el formulario muestra campos específicos como el peso y un checkbox para la agresividad. Si es un gato, se adapta para mostrar si es positivo en leucemia felina.



- Limpieza de Campos Automática: tras una utilización exitosa, los formularios se reestablecen a su estado inicial, listos para un nuevo uso sin dejar residuos de información anterior.

Resultados obtenidos:



Nombre	M	Sexo	Raza
Mia	12	hembra	Pastor
Edu	24	hembra	Pastor, Siberiano
Edu	26	hembra	Chow Chow Indivisa
Chel	27	macho	Chihuahua
Bruce	28	macho	Mixta Español

Tipo	Identificador	Hora Salida	Tiempo Inicial	Puntos Disponibles	Capacidad Total
PERRO	1	10:00	0	10	20
QUERRELA	2	10:00	0	10	20
AGRESIVO	3	10:00	0	10	20
AGRESIVO	4	10:00	0	10	20
AGRESIVO	5	10:00	0	10	20
AGRESIVO	6	10:00	0	10	20
AGRESIVO	7	10:00	0	10	20
AGRESIVO	8	10:00	0	10	20
AGRESIVO	9	10:00	0	10	20
AGRESIVO	10	10:00	0	10	20
AGRESIVO	11	10:00	0	10	20
AGRESIVO	12	10:00	0	10	20
CHIRL	13	10:00	0	10	20
CHIRL	14	10:00	0	10	20
CHIRL	15	10:00	0	10	20
CHIRL	16	10:00	0	10	20

3. Fase de Desarrollo y Arquitectura

3.1. Estructura y Tecnologías Utilizadas: detalles del entorno de desarrollo.

Para el desarrollo del proyecto he utilizado Java como lenguaje de programación, trabajando en el entorno de desarrollo IntelliJ IDEA Community Edition. El proyecto está configurado con Maven y JDK 21, lo que me ha permitido gestionar las dependencias y automatizar la compilación de la aplicación.

La interfaz gráfica la he desarrollado con JavaFX, utilizando las librerías javafx-controls y javafx-fxml. Además, para diseñar las ventanas y los archivos FXML he utilizado Scene Builder, ya que facilita mucho la creación de interfaces de manera más visual.

Para la base de datos he trabajado con MySQL a través de MySQL Workbench, permitiendo la gestión y creación de tablas de manera óptima.

El proyecto sigue el patrón de arquitectura **MVC (Modelo-Vista-Controlador)**, una estructura que permite separar la lógica de la aplicación en diferentes partes para mantener el código más ordenado y facilitar su mantenimiento.

Además, el proyecto incluye una capa DAO encargada de la base de datos y de realizar las consultas SQL necesarias.

3.2. Configuración del Proyecto y Persistencia:

3.2.1. Conexión a la Base de Datos

Para garantizar una gestión eficiente y segura de los recursos del sistema, el acceso a la base de datos se ha centralizado mediante la combinación del patrón Singleton y la persistencia basada en archivos de configuración XML procesados con JAXB.

- ConnectionBD: implementa el patrón Singleton para asegurar que exista una única instancia activa de la conexión durante todo el ciclo de vida de la ejecución.
- ConnectionProperties: almacena temporalmente los parámetros del servidor de la base de datos y recoge los métodos necesarios para recuperar las credenciales.
- XMLManager: una herramienta genérica y reutilizable basada en genéricos de Java. Proporciona la lógica necesaria para realizar operaciones de serializar (transformar objetos Java a estructuras XML) y deserializar (reconstruir objetos Java a partir de un flujo XML).

3.3. Planificación y Temporalización: sprints de desarrollo establecidos y calendario de cumplimiento.

Desarrollo del proyecto · 28 abril – 24 mayo

Sprint	Período	Fase	Tareas principales
Sprint 1	28 abr – 04 may	Análisis y Diseño	Definición de requisitos, diseño de la interfaz gráfica, diseño de entidad/relación y creación de la base de datos
Sprint 2	05 may – 10 may	Capa de Datos	Creación de los modelos y los DAOs con acceso completo a la base de datos
Sprint 3	11 may – 17 may	Lógica e Interfaz	Desarrollo de todas interfaces de usuario e implementación de controladores
Sprint 4	18 may – 21 may	Validación y Pruebas	Validación de entradas, gestión de errores y excepciones, pruebas funcionales totales.
Sprint 5	22 may – 24 may	Documentación	Redacción de la memoria técnica y revisión final del proyecto

4. Lista de Comprobación Técnica

4.1. Aplicación de la Programación Orientada a Objetos: Cómo se implementa la herencia, las restricciones con abstract y las llamadas a los constructores de la clase padre.

HERENCIA

La aplicación presenta dos jerarquías claras:

- Jerarquía de personas: *Persona* es la superclase, y tanto *Voluntario* como *Adoptante* extienden de ella sin añadir atributos propios. Su diferencia es que distinguen el rol de cada persona en el sistema.

```

11 public class Persona {
12     private String dni;
13     private String nombre;
14     private String apellidos;
15     private String telefono;
16     private String correo;
17     private String direccion;
18
19     /** Crea una nueva persona con todos sus datos personales. ... */
20     public Persona(String dni, String nombre, String apellidos, String telefono, String correo, String direccion) {
21         setDni(dni);
22         this.nombre = nombre;
23         this.apellidos = apellidos;
24         setTelefono(telefono);
25         this.correo = correo;
26         this.direccion = direccion;
27     }
28 }
    
```

Fragmento de la clase - Persona con sus atributos y constructor

```

8 public class Voluntario extends Persona {
9
10     /**
11      * Crea un nuevo voluntario con los datos personales indicados.
12      *
13      * @param dni --> NI o NIE del voluntario
14      * @param nombre --> nombre del voluntario
15      * @param apellidos --> apellidos del voluntario
16      * @param telefono --> teléfono de contacto
17      * @param correo --> dirección de correo electrónico
18      * @param direccion --> domicilio del voluntario
19      */
20     public Voluntario(String dni, String nombre, String apellidos, String telefono, String correo, String direccion) {
21         super(dni, nombre, apellidos, telefono, correo, direccion);
22     }
23
24     @Override
25     /**
26      * Devuelve una representación textual del voluntario.
27      * @return --> cadena de texto con el formato heredado Persona
28      */
29     public String toString() {
30         return super.toString();
31     }
32 }
    
```

Clase Voluntario, extiende de Persona


```

8 public class Adoptante extends Persona { 12 usages 1 Carmen Alcázar
9
10
11 /**
12  * Crea un nuevo adoptante con los datos personales indicados.
13  *
14  * @param dni --> DNI o NIE del adoptante
15  * @param nombre --> nombre del adoptante
16  * @param apellidos --> apellidos del adoptante
17  * @param telefono --> teléfono de contacto
18  * @param correo --> correo electrónico
19  * @param direccion --> domicilio del adoptante
20  */
21 public Adoptante(String dni, String nombre, String apellidos, String telefono, String correo, String direccion) {
22     super(dni, nombre, apellidos, telefono, correo, direccion);
23 }
24
25 /**
26  * @return --> cadena con el nombre del adoptante y su rol
27  */
28 public String toString() { return super.getNombre() + " es Adoptante"; }
29 }
30
31

```

Clase Adoptante, extiende de Persona

- Jerarquía de Animales: Animal es la superclase, por lo que Perro y Gato extienden de ella, añadiendo atributos específicos de cada especie (peso y agresivo en Perro; leucemiaFelina para Gato).

```

9
10 > /** Representa un animal del refugio con todos sus datos identificativos. */
11
12 public class Animal { 1 Carmen Alcázar
13
14     private int id; 8 usages
15     private String nombre; 9 usages
16     private String raza; 9 usages
17     private Sexo sexo; 9 usages
18     private String color; 7 usages
19     private String edad; 6 usages
20     private String marcasDistintivas; 6 usages
21     private String numeroChip; 6 usages
22     private boolean esterilizado = false; 7 usages
23     private String historia; 6 usages
24     private String observaciones; 6 usages
25     private LocalDate fechaIngreso; 5 usages
26     private boolean adoptado = false; 4 usages
27     private LocalDate fechaAlta; 6 usages
28     private String dniAdoptante; 5 usages
29     private int idUbicacion; 6 usages
30
31 > /** Constructor mínimo con los datos básicos de identificación. ...*/
32
33 public Animal(int id, String nombre, String raza, Sexo sexo) { 1 Carmen Alcázar
34     this.id = id;
35     this.nombre = nombre;
36     this.raza = raza;
37     this.sexo = sexo;
38 }
39
40
41

```

Fragmento de la clase Animal donde se ven sus atributos específicos y uno de sus constructores

```

public class Perro extends Animal implements Acogible {  & Carmen Alcázar

    private int idPerro; 3 usages
    private double peso; 6 usages
    private boolean agresivo; 8 usages

    /** Constructor completo usado por PerroDAO para reconstruir ... */
    public Perro(int id, String nombre, String raza, Sexo sexo, & Carmen Alcázar
        String color, String edad, String marcasDistintivas,
        String numeroChip, boolean esterilizado, String historia,
        String observaciones, LocalDate fechaIngreso, int idUbicacion,
        double peso, boolean agresivo) {

        super(id, nombre, raza, sexo, color, edad, marcasDistintivas, numeroChip,
            esterilizado, historia, observaciones, fechaIngreso, adoptado: false, fechaAlta: null, dniAdoptante: null, idUbicacion:
            this.peso = peso;
            this.agresivo = agresivo;
        }

    /** Constructor mínimo con solo los datos básicos de identificación. ... */
    public Perro(int id, String nombre, String raza, Sexo sexo) {  & Carmen Alcázar
        super(id, nombre, raza, sexo);
    }
}
    
```

Clase Perro, extiende de Animal

```

8 > /** Representa un gato del refugio. ... */
12 public class Gato extends Animal { 38 usages  & Carmen Alcázar
13     private boolean leucemiaFelina; 4 usages
14
15 > /** Constructor mínimo usado para operaciones básicas de referencia. ... */
24 public Gato(int id, String nombre, String raza, Sexo sexo, boolean leucemiaFelina) { 1 usage  & Carmen Alcázar
25     super(id, nombre, raza, sexo);
26     this.leucemiaFelina = leucemiaFelina;
27
28 }
29
30 > /** Constructor completo usado por GatoDAO para reconstruir ... */
39 public Gato(int id, String nombre, String raza, Sexo sexo, 1 usage  & Carmen Alcázar
40     String color, String edad,
41     String marcasDistintivas, String numeroChip, boolean esterilizado,
42     String historia, String observaciones, LocalDate fechaIngreso,
43     int idUbicacion, boolean leucemiaFelina) {
44     super(id, nombre, raza, sexo, color, edad, marcasDistintivas, numeroChip,
45         esterilizado, historia, observaciones, fechaIngreso, adoptado: false, fechaAlta: null, dniAdoptante: null, idUbicacion:
46         this.leucemiaFelina = leucemiaFelina;
47 }
48
    
```

Clase Gato, extiende de Animal

ABSTRACT E INTERFACES

En el proyecto no uso clases abstract directamente, pero la abstracción se aplica mediante la interfaz `Acogible`, que `Perro` implementa. Esta interfaz define el método `puedeSerAcogido()`, cuya lógica concreta se implementa en la clase `Perro`: un perro agresivo no puede estar en régimen de acogimiento.

```

1 package org.proyectorefugio.interfaces;
2
3 > /** Interfaz que deben implementar los animales susceptibles de ser acogidos ... */
9 public interface Acogible { 2 usages  & Carmen Alcázar
10     boolean puedeSerAcogido(); no usages 1 implementation  & Carmen Alcázar
11 }
12
    
```

Interfaz Acogible

```

135
136      @Override no usages  & Carmen Alcázar
137      /**
138       * Metodo que indica si un perro puede ser acogido no.
139       * Si es agresivo no puede ser acogido
140       *
141       * Este metodo existe para una futura ampliación del programa
142       * donde se contemple además de la Adopción, la Acogida de los animales
143       * a la espera de una familia que los adopte.
144       *
145       * @return --> devuelve false si no puede ser acogido
146       */
147      public boolean puedeSerAcogido() {
148          if(this.agresivo) {
149              return false;
150          }
151          return true;
152      }

```

Método puedeSerAcogido() en clase Perro

LLAMADAS AL CONSTRUCTOR DE LA CLASE PADRE

Cada subclase delega la inicialización de los atributos heredados al constructor de su superclase mediante `super()`. Esto evita duplicar código y garantiza que las **validaciones definidas en la superclase** (formato de DNI, teléfono, fechas, número de chip) se ejecuten siempre.

```

public Voluntario(String dni, String nombre, String apellidos, String telefono, String correo, String direccion) {
    super(dni, nombre, apellidos, telefono, correo, direccion);
}

```

Clase Voluntario

```

public Adoptante(String dni, String nombre, String apellidos, String telefono, String correo, String direccion) {
    super(dni, nombre, apellidos, telefono, correo, direccion);
}

```

Clase Adoptante

```

public Perro(int id, String nombre, String raza, Sexo sexo, & Carmen Alcázar
    String color, String edad, String marcasDistintivas,
    String numeroChip, boolean esterilizado, String historia,
    String observaciones, LocalDate fechaIngreso, int idUbicacion,
    double peso, boolean agresivo) {

    super(id, nombre, raza, sexo, color, edad, marcasDistintivas, numeroChip,
        esterilizado, historia, observaciones, fechaIngreso, adoptado: false, fechaAlta: null, dniAdoptante: null, idUbicacion);
    this.peso = peso;
    this.agresivo = agresivo;
}

/* Constructor mínimo con solo los datos básicos de identificación. ...*/
public Perro(int id, String nombre, String raza, Sexo sexo) { & Carmen Alcázar
    super(id, nombre, raza, sexo);
}

```

Clase Perro

```

public Gato(int id, String nombre, String raza, Sexo sexo, & Carmen Alcázar
    String color, String edad,
    String marcasDistintivas, String numeroChip, boolean esterilizado,
    String historia, String observaciones, LocalDate fechaIngreso,
    int idUbicacion, boolean leucemiaFelina) {
    super(id, nombre, raza, sexo, color, edad, marcasDistintivas, numeroChip,
        esterilizado, historia, observaciones, fechaIngreso, adoptado: false, fechaAlta: null, dniAdoptante: null, idUbicacion);
    this.leucemiaFelina = leucemiaFelina;
}

```

Clase Gato

4.2. Relaciones del Modelo de Datos: detalle de las relaciones utilizadas cumpliendo el mínimo de 4 tablas.

El modelo cuenta con 8 tablas en total, organizadas en tres bloques: persona, animal y ubicación.

Relación 1: persona → voluntario / adoptante (1:1 por herencia):

Una persona puede ser registrada como voluntario y como adoptante. Las tablas voluntario y adoptante no tienen atributos propios: sólo almacenan el DNI como clave primaria y clave foránea a persona al mismo tiempo

Relación 2 — animal → perro / gato (1:1 por herencia)

Las tablas perro y gato sólo almacenan los atributos propios de cada especie más el ID del animal como FK. El ID del animal en perro y gato es a la vez su clave primaria, lo que garantiza que no pueda existir un perro sin sus datos en animal base.

Relación 3 — adoptante → animal (1:N)

Un adoptante puede adoptar varios animales a lo largo del tiempo, pero cada animal sólo puede ser adoptado por una persona. La FK dniAdoptante se encuentra en la tabla animal, junto con el campo adoptado (boolean) y fechaAlta que registra cuándo abandonó el refugio.

Relación 4 — ubicacion → animal (1:N)

Cada animal está asignado a una ubicación física del refugio (jaula, chenil, zona de cuarentena o zona de agresivos). Una ubicación puede albergar varios animales hasta su capacidad máxima, pero cada animal pertenece a una sola ubicación en un periodo de tiempo, mientras no sea trasladado.

Relación 5 — voluntario ↔ ubicación a través de ayuda (N:M)

Un voluntario puede ayudar en varias ubicaciones distintas y, en una misma ubicación, pueden trabajar varios voluntarios. La tabla ayuda es la tabla intermedia que muestra esta relación, añadiendo además fecha y tarea como atributos propios de la relación, lo que la convierte en una **entidad asociativa**.

La clave primaria de *Ayuda* está compuesta por tres campos: (dniVoluntario, idUbicacion, fecha), lo que significa que un mismo voluntario puede ayudar en la misma ubicación en fechas distintas, pero no dos veces el mismo día en el mismo sitio.

4.3. Implementación del Patrón DAO: explicación de la construcción de los DAO y quién asume el control en las relaciones N:M.

CONSTRUCCIÓN DE LOS DAOS

Todos los DAO siguen exactamente el mismo esquema de construcción, lo que da consistencia a toda la capa de acceso a datos:

- Constantes SQL privadas y estáticas: todas las sentencias SQL se declaran al inicio de la clase como `private static final String`, agrupadas por regiones. Esto centraliza las consultas y evita SQL disperso por el código.

```
private final static String SQL_FIND_BY_DNI = "SELECT * FROM persona WHERE dni = ?"; 1usage
private final static String SQL_FIND_ALL = "SELECT * FROM persona"; 1usage
private final static String SQL_FIND_BY_NAME = "SELECT * FROM persona WHERE nombre LIKE ?"; 1usage
private final static String SQL_FIND_BY_LAST_NAME = "SELECT * FROM persona WHERE apellidos LIKE ?"; 1usage

private final static String SQL_INSERT = "INSERT INTO persona (dni, nombre, apellidos, telefono, correo, direccion) VALUES (?,?,?,?,?,?)"; 1usage

private final static String SQL_DELETE = "DELETE FROM persona WHERE dni = ?"; 1usage

private final static String SQL_UPDATE_PERSON = "UPDATE persona SET nombre=?, apellidos=?, telefono=?, correo=?, direccion=? WHERE dni = ?"; 1usage
private final static String SQL_UPDATE_TELEFONO = "UPDATE persona SET telefono = ? WHERE dni = ?"; 1usage
private final static String SQL_UPDATE_CORREO = "UPDATE persona SET correo = ? WHERE dni = ?"; 1usage
private final static String SQL_UPDATE_DIRECCION = "UPDATE persona SET direccion = ? WHERE dni = ?"; 1usage
```

Ejemplo de sentencias SQL, clase PersonaDAO

- Métodos estáticos: ningún DAO se instancia; todos sus métodos son `static`. No hay estado de objeto, sólo operaciones sobre la base de datos.
- Método `rellenar()` o `rellenarDatos()`: patrón propio para centralizar la construcción del objeto en un único punto. Si el modelo cambia (añadir un atributo), sólo hay que tocar ese método.

```

private static Perro rellenarDatosPerro(Animal a) {
    Perro p = null;
    try (PreparedStatement ps = ConnectionBD.getConnection().prepareStatement(SQL_FIND_PERRO)) {
        ps.setInt(1, a.getId());

        try (ResultSet rs = ps.executeQuery()) {
            if (rs.next()) {
                p = new Perro(
                    a.getId(),
                    a.getNombre(),
                    a.getRaza(),
                    a.getSexo(),
                    a.getColor(),
                    a.getEdad(),
                    a.getMarcasDistintivas(),
                    a.getNumeroChip(),
                    a.isEsterilizado(),
                    a.getHistoria(),
                    a.getObservaciones(),
                    a.getFechaIngreso(),
                    a.getIdUbicacion(),
                    rs.getDouble("peso"),
                    rs.getBoolean("agresivo")
                );
            }
        }
    } catch (SQLException e) {
        throw new RuntimeException(e);
    }
    return p;
}
    
```

Ejemplo en clase PerroDAO

DELEGACIÓN ENTRE DAOS

La aplicación también implementa herencia en base de datos mediante tablas separadas, y los DAOs resuelven esto con delegación explícita:

- Patrón en la jerarquía Persona: AdoptanteDAO y VoluntarioDAO no replican la lógica de lectura de persona. En cambio, delegan en PersonaDAO.findByDni() para obtener los datos comunes y sólo comprueban en su propia tabla si el DNI existe con esAdoptante() / esVoluntario().

```

public static List<Persona> findAll() {
    List<Persona> listaVoluntarios = new ArrayList<>();

    List<String> listaDNI = new ArrayList<>();
    try (ResultSet rs = ConnectionBD.getConnection().createStatement().executeQuery(SQL_FIND)) {
        while (rs.next()) {
            listaDNI.add(rs.getString("dniVoluntario"));
        }
    } catch (SQLException e) {
        throw new RuntimeException(e);
    }

    for (String dni : listaDNI) {
        Persona p = PersonaDAO.findByDni(dni);
        if (p != null) {
            listaVoluntarios.add(p);
        }
    }

    return listaVoluntarios;
}
    
```

Ejemplo en clase VoluntarioDAO

- Patrón en la jerarquía Animal: GatoDAO y PerroDAO delegan en AnimalDAO.findById() para obtener los 16 campos comunes, y luego lanzan una segunda consulta a su tabla propia (gato o perro) para obtener el atributo específico (leucemiaFelina, peso, agresivo).

```

38 @ public static List<Gato> findAll(boolean isAdopted) { // Carmen Alcázar
39     List<Gato> listaGatos = new ArrayList<>();
40     Gato gato = null;
41     try (PreparedStatement ps = ConnectionBD.getConnection().prepareStatement(SQL_FIND_ALL)) {
42         ps.setBoolean(1, isAdopted);
43
44         try (ResultSet rs = ps.executeQuery()) {
45             while (rs.next()) {
46                 int id = rs.getInt("id");
47                 Animal a = AnimalDAO.findById(id);
48
49                 gato = rellenarDatosGato(a);
50                 listaGatos.add(gato);
51             }
52         }
53     } catch (SQLException e) {
54         throw new RuntimeException(e);
55     }
56     return listaGatos;
57 }
    
```

Ejemplo en GatoDAO

RELACIÓN N:M

La única relación N:M es entre Voluntario y Ubicación, materializada en la tabla ayuda. Un voluntario puede ayudar en muchas ubicaciones, y en una ubicación pueden trabajar muchos voluntarios. El tercer atributo, fecha, convierte cada combinación de los tres atributos en un registro único.

AyudaDAO es el DAO que asume el control total de esta relación. VoluntarioDAO y UbicacionDAO no saben nada de la tabla ayuda; es AyudaDAO quien gestiona la clave primaria compuesta (dniVoluntario, idUbicacion, fecha) en todas sus operaciones.

Aspectos importantes de esta tabla y DAO:

- Consultas de navegación en ambas direcciones: AyudaDAO permite recorrer la relación desde cualquiera de los dos lados. findByDniVoluntario() va del voluntario hacia las ubicaciones, y findByIdUbicacion() va de la ubicación hacia los voluntarios.

```

public static List<Ayuda> findByDniVoluntario(String dni) { 3 usages  Carmen Alcázar
    List<Ayuda> tareasRelizadas = new ArrayList<>();
    Ayuda ayuda = null;
    try (PreparedStatement ps = ConnectionBD.getConnection().prepareStatement(SQL_FIND_BY_DNI)) {
        ps.setString(1, dni);
        ResultSet rs = ps.executeQuery();
        while (rs.next()) {
            int idUbicacion = rs.getInt("idUbicacion");
            LocalDate fecha = rs.getDate("fecha").toLocalDate();
            String tarea = rs.getString("tarea");

            String dniVoluntario = rs.getString("dniVoluntario");
            int idUbicacion = rs.getInt("idUbicacion");
            String tarea = rs.getString("tarea");
            ayuda = new Ayuda(dniVoluntario, idUbicacion, fecha, tarea);
            tareasRelizadas.add(ayuda);
        }
    } catch (SQLException e) {
        throw new RuntimeException(e);
    }
}
    
```

findByDniVoluntario()

- Los UPDATE modifican partes de la PK: `updateUbicacion()` y `updateFecha()` cambian campos que forman parte de la clave primaria, lo que en una relación N:M requiere especial cuidado. El DAO lo resuelve usando los tres valores originales en el WHERE y el nuevo valor en el SET.

```

public static boolean updateUbicacion(Ayuda a, int idUbicacion) { 1 usage  Carmen Alcázar
    boolean updated = false;
    if ((a != null) && findSingle(a.getDniVoluntario(), a.getIdUbicacion(), a.getFecha()) != null) {
        try (PreparedStatement ps = ConnectionBD.getConnection().prepareStatement(SQL_UPDATE_UBICACION)) {
            ps.setInt(1, idUbicacion);
            ps.setString(2, a.getDniVoluntario());
            ps.setInt(3, a.getIdUbicacion());
            ps.setDate(4, Date.valueOf(a.getFecha()));

            int filasAfectadas = ps.executeUpdate();
            updated = (filasAfectadas > 0);

            updated = true;
        } catch (SQLException e) {
            throw new RuntimeException(e);
        }
    }
    return updated;
}
    
```

updateUbicacion()

4.4. Operaciones CRUD Básicas: mecanismo y flujo de las funciones Mostrar, Insertar y Eliminar.

FLUJO GENERAL DE TODAS LAS OPERACIONES

Todas las operaciones siguen el mismo esqueleto de tres pasos: obtener la conexión mediante la clase `ConnectionBD` con el método `getConnection()`, preparar la sentencia con `PreparedStatement`, y gestionar el resultado.

MOSTRAR (FIND)

Estrategias de carga: Eager y Lazy

Lazy loading (carga perezosa) consiste en recuperar únicamente los datos mínimos necesarios en el momento de la consulta, dejando el resto sin cargar hasta que se necesite. Eager loading (carga ansiosa) consiste en recuperar todos los datos del objeto de golpe en la misma operación, aunque no todos vayan a usarse de inmediato.

- LAZY → `PerroDAO.findAll()`: la consulta SQL selecciona solo 4 columnas (id, nombre, raza, sexo) y construye cada objeto con el constructor mínimo `Perro(id, nombre, raza, sexo)`. El resto de campos no se cargan. Este método es suficiente para mostrar los datos básicos en la tabla del fxml.

```
private final static String SQL_FIND_ALL = "SELECT a.id, a.nombre, a.raza, a.sexo FROM animal a, perro p WHERE a.id = p.idPerro AND adoptado = ?";
```

```
public static List<Perro> findAll(boolean isAdopted) { //LAZY & Carmen Alcázar *
    List<Perro> listaPerros = new ArrayList<>();
    Perro perro = null;
    try (PreparedStatement ps = ConnectionBD.getConnection().prepareStatement(SQL_FIND_ALL)) {
        ps.setBoolean(1, isAdopted);

        try (ResultSet rs = ps.executeQuery()) {
            while (rs.next()) {
                int id = rs.getInt(columnLabel: "id");
                String nombre = rs.getString(columnLabel: "nombre");
                String raza = rs.getString(columnLabel: "raza");
                Sexo sexo = Sexo.valueOf(rs.getString(columnLabel: "sexo"));

                perro = new Perro(id, nombre, raza, sexo);

                listaPerros.add(perro);
            }
        }
    } catch (SQLException e) {
        throw new RuntimeException(e);
    }
    return listaPerros;
}
```

- EAGER → Todos los métodos `findById()` y `findByDni()`: cuando busca un registro concreto, se carga su ficha completa de inmediato.

```
private final static String SQL_FIND_BY_DNI = "SELECT * FROM persona WHERE dni = ?";
```

```

public static Persona findByDni(String dni) { 16 usages  Carmen Alcázar
    Persona p = null;

    try (PreparedStatement ps = ConnectionBD.getConnection().prepareStatement(SQL_FIND_BY_DNI)) {
        ps.setString(1, dni);
        ResultSet rs = ps.executeQuery();
        while (rs.next()) {
            String nombre = rs.getString(columnLabel: "nombre");
            String apellidos = rs.getString(columnLabel: "apellidos");
            String telefono = rs.getString(columnLabel: "telefono");
            String correo = rs.getString(columnLabel: "correo");
            String direccion = rs.getString(columnLabel: "direccion");

            p = new Persona(dni, nombre, apellidos, telefono, correo, direccion);
        }
    } catch (SQLException e) {
        throw new RuntimeException(e);
    }
    return p;
}
    
```

INSERTAR (INSERT)

El mecanismo de inserción también presenta dos comportamientos según si la entidad necesita devolver el objeto creado o sólo confirmar el éxito:

- Devuelven el objeto creado: `PersonaDAO.addPersona()` y `AyudaDAO.addAyuda()` insertan el registro y acto seguido lanzan un `findByDni()` / `findSingle()` para devolver el objeto tal y como quedó en la base de datos. Esto garantiza que el objeto devuelto refleja el estado real de la Base de Datos.

```

public static Persona addPersona(Persona p){ 1 usage  Carmen Alcázar
    Persona añadida = null;
    if(p!=null){
        try(PreparedStatement ps = ConnectionBD.getConnection().prepareStatement(SQL_INSERT)){
            ps.setString(1, p.getDni());
            ps.setString(2, p.getNombre());
            ps.setString(3, p.getApellidos());
            ps.setString(4, p.getTelefono());
            ps.setString(5, p.getCorreo());
            ps.setString(6, p.getDireccion());

            ps.executeUpdate();

            añadida = findByDni(p.getDni());
        } catch (SQLException e){
            throw new RuntimeException(e);
        }
    }
    return añadida;
}
    
```

Clase PersonaDAO

- Devuelven un boolean: AdoptanteDAO.addAdoptante(), VoluntarioDAO.addVoluntario(), GatoDAO.addGato() y PerroDAO.addPerro() sólo insertan la parte específica de la entidad (el rol o los atributos propios), ya que la parte base (persona o animal) se habrá insertado antes. Devuelven true o false.

```
public static boolean addAdoptante(Persona p) {
    if ((p != null)) {
        try (PreparedStatement ps = ConnectionBD.getConnection().prepareStatement(SQL_INSERT)) {
            ps.setString(1, p.getUn());
            ps.executeUpdate();
            return true;
        } catch (SQLException e) {
            throw new RuntimeException(e);
        }
    }
    return false;
}
```

Clase AdoptanteDAO

- Devuelven un int: AnimalDAO.addAnimal() devuelve el valor del último ID insertado para que las FK pueda insertar a la vez su correspondiente Perro o Gato

```
public static int addAnimal(Animal animal) {
    Animal añadido = null;
    if ((animal != null) && findByChip(animal.getNumeroChip()) == null) {
        try (PreparedStatement ps = ConnectionBD.getConnection().prepareStatement(SQL_INSERT_ANIMAL, Statement.RETURN_GENERATED_KEYS)) {
            ps.setString(1, animal.getNombre());
            ps.setString(2, animal.getRaza());
            ps.setString(3, animal.getSexo().toLowerCase());
            ps.setString(4, animal.getColor());
            ps.setString(5, animal.getEdad());
            ps.setString(6, animal.getMarcasDistintivas());
            ps.setString(7, animal.getNumeroChip());
            ps.setBoolean(8, animal.isEsterilizado());
            ps.setString(9, animal.getHistoria());
            ps.setString(10, animal.getObservaciones());
            ps.setInt(11, animal.getIdUbicacion());
            ps.setDate(12, Date.valueOf(animal.getFechaIngreso()));
            ps.executeUpdate();
            try (ResultSet rs = ps.getGeneratedKeys()) {
                if (rs.next()) {
                    return rs.getInt(1);
                }
            }
        } catch (SQLException e) {
            throw new RuntimeException(e);
        }
    }
    return -1;
}
```

Clase AnimalDAO

ELIMINAR (DELETE)

El flujo de eliminación es uniforme en todos los DAOs que lo implementan. Primero se verifica que el registro existe llamando al método `find` correspondiente. Si existe, se ejecuta el `DELETE` con `PreparedStatement`. Finalmente, se devuelve `true` si se eliminó correctamente, `false` si el registro no existía.

```
public static boolean deletePersona(String dni) { 1 usage  Carmen Alcázar
    if (findByDni(dni) != null) {
        try (PreparedStatement ps = ConnectionBD.getConnection().prepareStatement(SQL_DELETE)) {
            ps.setString(1, dni);
            ps.executeUpdate();
            return true;
        } catch (SQLException e) {
            throw new RuntimeException(e);
        }
    }
    return false;
}
```

Clase PersonaDAO

4.5. Operaciones CRUD Avanzadas: Mecanismo de actualización y control de impedancia Relación-Objeto.

UPDATE

El mecanismo de actualización sigue siempre tres pasos antes de ejecutar la sentencia: comprobar que el objeto recibido no es null y comprobar que el registro existe en la Base de Datos con un `find` previo. Sólo si ambas condiciones se cumplen, ejecutar el `UPDATE`.

El resultado se obtiene mediante `ps.executeUpdate()` que devuelve el número de filas afectadas, y se convierte en boolean con `updated = (filasAfectadas > 0)`.

```
public static boolean updateNumeroChip(Animal a, String numeroChip) { 2 usages  Carmen Alcázar
    boolean updated = false;
    if ((a != null) && findByID(a.getId()) != null && findByChip(numeroChip) == null) {
        try (PreparedStatement ps = ConnectionBD.getConnection().prepareStatement(SQL_UPDATE_CHIP)) {
            ps.setString(1, numeroChip);
            ps.setInt(2, a.getId());

            int filasAfectadas = ps.executeUpdate();
            updated = (filasAfectadas > 0);
        } catch (SQLException e) {
            throw new RuntimeException(e);
        }
    }
    return updated;
}
```

Clase AnimalDAO

CONTROL DE IMPEDANCIA RELACIÓN-OBJETO**Impedancia 1 — Conversión de fechas y horas**

SQL trabaja con `DATE` y `TIME`, sin embargo, Java trabaja con `LocalDate` y `LocalTime`. La conversión es obligatoria en ambas direcciones y no es automática.

En lectura: `rs.getDate("fecha").toLocalDate()` y `rs.getTime("horaRecreo").toLocalTime()`.

En escritura: `Date.valueOf(a.getFecha())` y `Time.valueOf(horaRecreo)`.

```
ps.setDate( parameterIndex: 3, Date.valueOf(a.getFecha()));
```

AyudaDAO.addAyuda()

```
//Convierte el tipo de dato Time que recibe de la base
// de datos en tipo de dato LocalTime con el que trabaja Java
LocalTime hora = null;
if (horaRecreo != null) {
    hora = horaRecreo.toLocalTime();
}
```

UbicacionDAO.fingById()

Impedancia 2 — Conversión de enumerados

Java usa `enum` (`Sexo`, `Ubicaciones`) para representar conjuntos cerrados de valores; SQL los almacena como `VARCHAR`. La conversión se hace en ambas direcciones con `Enum.valueOf()` al leer y `.toString()` al escribir.

```
Ubicaciones tipo = Ubicaciones.valueOf(rs.getString( columnLabel: "tipo").toUpperCase());
```

UbicacionDAO.findById()

```
Sexo sexo = Sexo.valueOf(rs.getString( columnLabel: "sexo"));
```

PerroDAO.findAll()

Impedancia 3 — Herencia de objetos y tablas SQL

La herencia de Java no existe en SQL. Este proyecto la resuelve con una tabla por subclase: cada nivel de herencia tiene su propia tabla, y los DAOs hijos hacen siempre dos consultas para reconstruir el objeto completo.

- `GatoDAO` → `rellenarDatosGato(Animal a)`: recibe el objeto `Animal` ya construido y lo combina con los datos de la tabla `gato` para devolver un `Gato` completo.

5. Valores Añadidos y Calidad del Código

5.1. Validaciones y Control de Errores: control de campos protegidos, controles desactivados y muestra de mensajes de error gráficos.

VALIDACIÓN EN EL MODELO

Utils centraliza todas las validaciones de formato como métodos estáticos reutilizables. Los setters de las clases modelo (`Persona.setDni()`, `Animal.setNumeroChip()`, `Animal.setFechaIngreso()`) llaman directamente a estos métodos y lanzan `IllegalArgumentException` si el formato no es válido, impidiendo que se cree un objeto con datos incorrectos.

```
public static boolean validarDNI(String dni) { 2 usages  Carmen Alcázar
    if ((dni != null) && (dni.matches(regex: "[0-9]{8}[A-Z]") || dni.matches(regex: "[XYZ][0-9]{7}[A-Z]"))) {
        return true;
    }
    return false;
}
```

Clase Utils

```
public void setDni(String dni) {  Carmen Alcázar
    String dniMayuscula = dni.toUpperCase().trim();
    if (dniMayuscula.length() != 9) {
        throw new IllegalArgumentException("DNI/NIE no válido: debe tener 9 caracteres");
    }

    if (!Utils.validarDNI(dniMayuscula)) {
        throw new IllegalArgumentException("DNI/NIE no válido");
    }
    this.dni = dniMayuscula;
}
```

Clase Persona

VALIDACIÓN EN LOS CONTROLADORES

- Validación de campos obligatorios vacíos: todos los formularios comprueban que los campos requeridos no estén vacíos antes de intentar ninguna operación. Usa `.trim().isEmpty()` para detectar también campos con solo espacios.

```
if (idAnimalTexto.isEmpty() && chipAnimal.isEmpty() &&
    nombreAnimal.isEmpty() && razaAnimal.isEmpty() && colorAnimal.isEmpty()) {
    Mensajes.alertaObligatoriosCamposVacios(mensaje: "Introduce al menos un criterio de búsqueda");
    tablaGatos();
    return null;
}
```

GatoController.buscarAnimal()

LISTENER EN TIEMPO REAL SOBRE EL CAMPO DNI

En el formulario de registro, el campo DNI tiene un `textProperty().addListener()` que se activa con cada tecla pulsada. Si el DNI introducido ya existe en la base de datos, rellena automáticamente el resto del formulario con los datos de esa persona, evitando duplicados y errores de tecleo.

```
public void initialize() { // Carmen Alcázar
    definirTipoPersona();
    infoDNI.textProperty().addListener((ObservableValue<extends String> observable, String oldValue, String newValue) -> {
        Persona p = PersonaDAO.findByDni(newValue);
        if(p!=null){
            infoNombre.setText(p.getNombre());
            infoApellidos.setText(p.getApellidos());
            infoTelefono.setText(p.getTelefono());
            infoCorreo.setText(p.getCorreo());
            infoDireccion.setText(p.getDireccion());
        }
    });
}
```

Clase FormularioPersonaYAdoptarController

CONTROL DE EXISTENCIA PREVIA ANTES DE INSERTAR/ELIMINAR

En `guardarInformacion()` de `FormularioPersonaYAdoptarController`: si la persona ya existe se llama a `updatePerson()`, si no existe se llama a `addPersona()`.

```
public void guardarInformacion(ActionEvent event) {
    Persona registrar = obtenerInformacionPersonaDelFormulario();
    if(registrar==null){
        return;
    }
    if (PersonaDAO.findByDni(registrar.getDni()) != null) {
        PersonaDAO.updatePerson(registrar);
    } else {
        PersonaDAO.addPersona(registrar);
    }

    if ("adoptante".equals(persona)) {
        if (AdoptanteDAO.findByDni(registrar.getDni()) == null) {
            AdoptanteDAO.addAdoptante(registrar);
        }

        Animal a = AnimalDAO.findById(Utils.conversorInt(infoIdAnimal.getText()));
        if (a != null) {
            AnimalDAO.updateAdoptante(a, registrar.getDni());
            AnimalDAO.updateFechaAlta(a, LocalDate.now());
        }
    }

    if ("voluntario".equals(persona)) {
        if (VoluntarioDAO.findByDni(registrar.getDni()) == null) {
            VoluntarioDAO.addVoluntario(registrar);
        }
    }

    limpiarCampos();

    Stage stage = (Stage) botonCancelar.getScene().getWindow();
    stage.close();
    Mensajes.operacionCompletada(mensaje: "Registro completado");
}
```


En `botonEliminar()` de `PersonaController`: antes de eliminar comprueba si la persona es adoptante con un animal asociado o voluntaria con tareas pendientes. Si tiene dependencias, muestra el aviso y bloquea el borrado.

```
public void botonEliminar(ActionEvent event) {
    informacionAdicional.setVisible(false);
    ventanaBuscar.setVisible(false);
    Persona persona = tablaPersonas.getSelectionModel().getSelectedItem();

    if (persona == null) {
        Mensajes.alertaNoSeleccionado( mensaje: "No hay ningún elemento seleccionado");
        return;
    }

    Voluntario v = VoluntarioDAO.findById(persona.getId());
    Adoptante a = AdoptanteDAO.findById(persona.getId());

    if (v != null || a != null) {
        if (a != null) {
            Mensajes.noPuedeEliminar( mensaje: "No puedes eliminar a esta persona porque tiene un animal adoptado");
        } else {
            Mensajes.noPuedeEliminar( mensaje: "No puedes eliminar a esta persona porque tiene tareas asociadas");
        }
    }

    if (Mensajes.confirmarEliminar( mensaje: "¿Desea eliminar el elemento de forma permanente?")) {
        PersonaDAO.deletePersona(persona.getId());
    }

    initialize();
}
```

VALIDACIÓN DE SELECCIÓN EN TABLA

Antes de ejecutar modificar o eliminar, se verifica que haya un elemento seleccionado en la `TableView`. Si no hay selección, muestra un aviso y aborta la operación.

```
public boolean modificarPersona() {
    Persona p = tablaPersonas.getSelectionModel().getSelectedItem();
    if (p == null) {
        Mensajes.alertaNoSeleccionado( mensaje: "Debe seleccionar al menos un elemento");
        return false;
    }
}
```

Fragmento

FORMULARIOS DINÁMICOS

Los formularios adaptan los campos visibles según el contexto, ocultando controles que no aplican. Esto evita que el usuario pueda introducir datos incorrectos en campos que no corresponden a su caso.

5.2. Adaptabilidad de la Interfaz: ventanas redimensionables y comportamiento adaptativo.

VENTANA PRINCIPAL REDIMENSIONABLE CON TAMAÑO MÍNIMO

`PrincipalApplication` configura la ventana raíz con `setResizable(true)` para permitir que el usuario la redimensione libremente, y establece un tamaño mínimo de 1500×780 px con `setMinWidth()` y `setMinHeight()`.

```
public void start(Stage stage) throws IOException {
    FXMLLoader fxmlLoader = new FXMLLoader(PrincipalApplication.class.getResource("main.fxml"));
    Scene scene = new Scene(fxmlLoader.load(), 1500, 780);
    stage.setMinWidth(1500);
    stage.setMinHeight(780);
    stage.setResizable(true);
    stage.setTitle("Refugio");
    stage.setScene(scene);
    stage.show();
}
```

NAVEGACIÓN DINÁMICA SIN CAMBIO DE VENTANA

`PrincipalController` gestiona toda la navegación de la aplicación cargando las distintas vistas dentro de un mismo `panelContenido` mediante `SceneManager.cargarVista()`. No se abren nuevas ventanas para el contenido principal: la ventana raíz permanece fija y solo cambia el contenido interior

PANELES INTERNOS CON VISIBILIDAD CONTROLADA

`PersonaController` gestiona tres paneles internos (`ventanaBuscar`, `informacionAdicional`, `panelModificacion`) que aparecen y desaparecen según la acción en curso. El método `ocultarTodosPaneles()` se llama en `initialize()` para garantizar un estado limpio al cargar la vista, y cada botón de acción oculta los paneles que no corresponden antes de mostrar el suyo.

VENTANAS EMERGENTES NO REDIMENSIONABLES

Los formularios de registro y adopción se abren como ventanas emergentes secundarias con `setResizable(false)` (tamaño fijo, no adaptable).

5.3. Características Avanzadas (Streams y Lambdas): Uso de características modernas de Java en el tratamiento de datos.

Se hace uso de lambdas en tres contextos distintos: listeners de propiedades reactivas, fábricas de celdas para tablas y gestión de eventos.

- En `FormularioPersonaYAdoptarController.initialize()`, el campo DNI tiene un `ChangeListener` implementado como lambda que se dispara con cada pulsación de tecla. Cuando el valor nuevo corresponde a un DNI existente en la base de datos, rellena automáticamente el resto del formulario.

```
public void initialize() {
    definirTipoPersona();
    infoDNI.textProperty().addListener((ObservableValue extends String observable, String oldValue, String newValue) -> {
        Persona p = PersonaDAO.findById(newValue);
        if(p!=null){
            infoNombre.setText(p.getNombre());
            infoApellidos.setText(p.getApellidos());
            infoTelefono.setText(p.getTelefono());
            infoCorreo.setText(p.getCorreo());
            infoDireccion.setText(p.getDireccion());
        }
    });
}
```

- En `PersonaController.mostrarInformacionAdicional()`, la selección en la `TableView` tiene un `ChangeListener` como lambda que se activa cada vez que el usuario hace clic en una fila. Oculta el panel de búsqueda, hace visible el `Label` de información adicional y construye el texto con los datos de la persona seleccionada.

```
public void mostrarInformacionAdicional() {
    tablaPersonas.getSelectionModel().selectedItemProperty().addListener((ObservableValue extends Persona observable, Persona anterior, Persona seleccionado) -> {
        if (seleccionado != null) {
            ventanaBuscar.setVisible(false);
            informacionAdicional.setVisible(true);
            informacionAdicional.setText(
                "Nombre: " + seleccionado.getNombre() + "\n" +
                "DNI: " + seleccionado.getDni() + "\n" +
                "Telefono: " + seleccionado.getTelefono() + "\n" +
                "Correo: " + seleccionado.getCorreo() + "\n" +
                "Direccion: " + seleccionado.getDireccion()
            );
        }
    });
}
```

- En `PersonaController.tablaPersonas()`, la columna `nombreCompletoCol` no mapea directamente un campo del objeto, sino que nombre + apellidos en una sola celda usando una lambda como `Callback`. Devuelve un `SimpleStringProperty` construido con la concatenación, algo que `PropertyValueFactory` no permite por sí solo.

```

public void tablaPersonas() {
    nombreCompletoCol.setCellValueFactory(
        new CellDataFeatures<Persona, String>() {
            @Override public String getCellData(
                ObservableList<Persona> data,
                int index) {
                Persona p = data.get(index);
                return new SimpleStringProperty(
                    p.getNombre() + " " + p.getApellidos());
            }
        });

    dniCol.setCellValueFactory(new PropertyValueFactory<>("dni"));

    ObservableList<Persona> lista;

    if (voluntario.isSelected()) {
        lista = FXCollections.observableArrayList(VoluntarioDAO.findAll());
        tablaPersonas.setItems(lista);
    } else {
        lista = FXCollections.observableArrayList(AdoptanteDAO.findAll());
        tablaPersonas.setItems(lista);
    }
}
    
```

5.4. Gestión de Datos Complejos: tratamiento de fechas, expresiones regulares o datos multimedia.

GESTIÓN DE FECHAS CON LOCALDATE

Uso de `LocalDate` de la API `java.time` para todas las fechas, evitando la clase obsoleta `java.util.Date`

CAPTURA DE FECHA CON VALOR POR DEFECTO

En `RegistroAnimalController.obtenerInformacionGenericaDelFormulario()`, la fecha de ingreso procede de un `DatePicker`. Si el usuario no selecciona fecha, `getValue()` devuelve `null` y el controlador asigna `LocalDate.now()` como valor por defecto, garantizando que el campo nunca llega al DAO como nulo.

ASIGNACIÓN DE FECHA DE ADOPCIÓN

En `FormularioPersonaYAdoptarController.guardarInformacion()`, la fecha de ingreso del animal se establece con `LocalDate.now()` en el momento exacto de confirmar la adopción, sin intervención del usuario.

```

    LocalDate fechaIngreso = infoFecha.getValue();
    if (fechaIngreso == null) {
        fechaIngreso = LocalDate.now();
    }
    
```

VALIDACIÓN DE QUE LA FECHA NO SEA FUTURA

`Utils.validarFecha()` comprueba con `!fecha.isAfter(LocalDate.now())` que ninguna fecha de ingreso o alta sea posterior al día actual.

```
public static boolean validarFecha(LocalDate fecha) { 3 usages  Carmen Alcázar
    if (fecha == null) {
        return false;
    }
    return !fecha.isAfter(LocalDate.now());
}
```

EXPRESIONES REGULARES

Uso de expresiones regulares para verificar que un DNI es correcto, incluyendo también la versión NIE para personas extranjeras. También se verifica el número de teléfono para que no se puedan añadir caracteres, incluyendo también la versión con el prefijo internacional y espacios.

Validación del número chip ya que tiene un formato fijo de 15 dígitos. También valida el formato de la hora, tanto para 09:00 como 9:00.

```
public static boolean validarDNI(String dni) { 2 usages  Carmen Alcázar

    if ((dni != null) && (dni.matches(regex: "[0-9]{8}[A-Z]") || dni.matches(regex: "[XYZ][0-9]{7}[A-Z]"))) {
        return true;
    }
    return false;
}
```

```
public static LocalTime validarHora(String hora) { 3 usages  Carmen Alcázar
    if (hora == null) {
        return null;
    }
    if (hora.matches(regex: "([01]?[0-9]|2[0-3]):([0-5][0-9])")) {
        return LocalTime.parse(hora, DateTimeFormatter.ofPattern("H:mm"));
    }
    return null;
}
```

6. Consideraciones Futuras y Conclusión

6.1. Aspectos a mejorar de cara a una futura versión.

REFACTORIZACIÓN DEL CÓDIGO

Como he ido añadiendo funcionalidades de forma iterativa según surgían las necesidades del refugio, algunos controladores han acabado con métodos excesivamente largos que asumen demasiadas responsabilidades. Sería conveniente extraer la lógica repetida en clases de servicio intermedias entre los controladores y los DAO, y revisar las llamadas encadenadas a la base de datos para reducir el uso de recursos. Al estar aprendiendo, la resolución de errores y el *debugging* me consumían muchísimo tiempo; por eso, una vez que conseguía que una lógica funcionase correctamente, optaba por implementarla directamente en las demás clases para asegurar la funcionalidad del proyecto.

RÉGIMEN DE ACOGIDA PARA MASCOTAS ANTES DE LA ADOPCIÓN

Actualmente la aplicación gestiona la transición directa de un animal desde el refugio a su adopción definitiva. Sin embargo, en muchas protectoras reales existe una fase intermedia conocida como régimen de acogida temporal, en la que una familia alberga al animal durante un periodo de adaptación antes de la adopción oficial.

Una mejora relevante sería implementar este flujo en el sistema: incorporar una nueva entidad Acogida vinculada al animal y a la persona acogedora. La interfaz Acogible ya definida en el proyecto constituye la base perfecta sobre la que construir esta funcionalidad.

MEJORA DE LA INTERFAZ GRÁFICA CON CSS

La interfaz actual funciona correctamente y es funcional, pero su aspecto visual es básico al no aplicar hojas de estilo personalizadas. JavaFX permite adjuntar archivos CSS, lo que abriría la puerta a una mejora visual, ya que muchas cosas están con su diseño por defecto de SceneBuilder

OTRAS MEJORAS

La implementación de un sistema de notificaciones o recordatorios para gestionar próximas revisiones veterinarias o vencimientos del periodo de acogida.

6.2. Conclusiones y aprendizaje.

Este proyecto ha supuesto un reto personal significativo, ya que ha exigido aplicar de forma integrada prácticamente todos los contenidos trabajados a lo largo del curso. El hecho de haber partido de un problema real que a mí me llega mucho, ha sido una motivación constante durante el desarrollo, ya que en todo momento era posible visualizar dónde y cómo encajaba cada pieza.

Para mí, esta causa no es nueva; ya había colaborado y realizado otros proyectos anteriores con protectoras de animales en el ámbito fotográfico, por lo que poder aportar ahora una solución desde el lado del desarrollo de software ha hecho que este proyecto sea todavía más especial y cercano.

A nivel de proceso, este proyecto ha reforzado la importancia de una buena planificación inicial. La dedicación del primer sprint al diseño de la base de datos y el diagrama de clases antes de escribir una sola línea de código fue una decisión acertada, me quitó de encima muchísimas refactorizaciones y horas de corregir código roto después. Sin embargo, también ha quedado de manifiesto que en proyectos de esta envergadura siempre aparecen imprevistos que obligan a revisar decisiones tomadas con anterioridad, y que la capacidad de adaptación es tan importante como el diseño previo.

En definitiva, este proyecto demuestra que una aplicación de escritorio desarrollada con Java, JavaFX y MySQL puede dar respuesta real y profesional a las necesidades de gestión de una organización sin ánimo de lucro.